

Infrastructure Requirements

PR-DR Automated Deployment System

□ Overview

This document outlines the complete infrastructure requirements for implementing an automated PR (Production) and DR (Disaster Recovery) deployment system. The architecture uses Jenkins for orchestration and Ansible for configuration management to deploy Java microservices across multiple environments.

Key Benefits:

- □ **Zero-downtime deployments** with automated rollback capability
 - □ **Consistent deployments** across PR and DR environments
 - □ **Reduced human error** through automation
 - □ **Faster recovery** in case of failures
 - □ **Audit trail** of all deployment activities
-

□ Control Node (Deployment Server)

□ Control Node (Deployment Server)

Role: Central orchestration hub that manages all deployment activities

The Control Node is the brain of the deployment system. It doesn't run any business applications but manages the entire deployment lifecycle from a single location.

Services Running:

1. ▯ Jenkins (CI/CD Automation)

Purpose: Orchestrates the entire deployment pipeline and provides visibility

What it does:

- **Triggers deployments** based on schedules (e.g., every night at 2 AM) or manual triggers
- **Manages workflow** - Deploys to PR first, validates, then deploys to DR
- **Provides web UI** for monitoring deployment status in real-time
- **Stores history** - Complete audit trail of all deployments with logs
- **Handles failures** - Automatically triggers rollback if deployment fails
- **Sends notifications** - Email/Slack alerts on deployment success/failure
- **Manages permissions** - Controls who can trigger deployments

Why Jenkins?

- Industry-standard CI/CD tool with proven reliability
- Rich plugin ecosystem for integrations
- Easy-to-use web interface for non-technical users
- Supports complex pipeline workflows
- Free and open-source

Example Use Case:

When a new JAR file is ready, Jenkins automatically:

1. Backs up current version
2. Deploys to PR servers
3. Validates deployment
4. If successful, deploys to DR
5. Sends success notification

2. ▯ Ansible (Deployment Orchestration)

Purpose: Executes actual deployment tasks on remote servers

What it does:

- **Connects remotely** - Uses SSH to securely connect to all PR/DR servers
- **Stops services** - Gracefully stops Java applications before deployment
- **Backs up files** - Creates backup of existing JAR files (for rollback)
- **Deploys new code** - Copies new JAR files to target servers
- **Updates configs** - Manages environment-specific configurations
- **Starts services** - Restarts applications with proper JVM settings
- **Validates health** - Checks if services started successfully

- **Performs rollbacks** - Restores backup JARs if deployment fails
- **Parallel execution** - Deploys to multiple servers simultaneously

Why Ansible?

- Agentless - No software needed on target servers
- Simple YAML syntax - Easy to read and maintain
- Idempotent - Safe to run multiple times
- Built-in modules for common tasks
- Excellent for configuration management

Example Playbook Task:

```
- name: Deploy new JAR file
  copy:
    src: /path/to/new/app.jar
    dest: /opt/app/app.jar
    backup: yes
```

3. ■ Python 3.x

Purpose: Runtime environment for Ansible and custom automation scripts

What it does:

- **Runs Ansible** - Ansible is built on Python
- **Executes scripts** - Custom pre/post-deployment automation
- **Processes data** - Validates configurations, parses logs
- **API interactions** - Communicates with external systems

Why Python?

- Required by Ansible
- Versatile scripting language
- Rich library ecosystem
- Easy to write automation scripts

4. ■ SSH Client

Purpose: Secure remote access to PR/DR servers

What it does:

- **Establishes connections** - Creates encrypted tunnels to remote servers
- **Authenticates** - Uses SSH keys (no passwords needed)
- **Executes commands** - Runs deployment commands remotely
- **Transfers files** - Securely copies JAR files to servers

Why SSH?

- Industry-standard secure protocol
- Key-based authentication is more secure than passwords
- Encrypted communication
- Supports automation (no interactive prompts)

Setup Requirement:

SSH keys must be configured from Control Node to all PR/DR servers for passwordless authentication.

5. Git (Optional but Recommended)

Purpose: Version control for deployment artifacts and configurations

What it does:

- **Tracks changes** - Version history of playbooks, scripts, configs
- **Pulls updates** - Fetches latest deployment code from repositories
- **Maintains history** - Who changed what and when
- **Enables collaboration** - Multiple team members can contribute

Why Git?

- Industry-standard version control
- Easy rollback to previous configurations
- Collaboration and code review
- Integration with Jenkins

Files Stored on Control Node:

File Type	Purpose	Example
Playbooks	Ansible deployment scripts	<code>deploy.yml</code> , <code>rollback.yml</code> , <code>validate.yml</code>
Scripts	Custom automation	<code>pre_deploy.sh</code> , <code>health_check.py</code>
Configurations	Environment variables	<code>pr_vars.yml</code> , <code>dr_vars.yml</code>
Inventory	Server lists	<code>server_ips.yml</code>
Pipeline	Jenkins workflow	<code>Jenkinsfile</code>
SSH Keys	Authentication	<code>id_rsa</code> , <code>id_rsa.pub</code>

☐ Resource Requirements:

Resource	Minimum	Recommended	Why?
Memory	8 GB	16 GB	Jenkins + Ansible need memory for parallel operations
Disk	20 GB	50 GB	Store logs, backups, and deployment artifacts
CPU	2 cores	4 cores	Handle concurrent deployments
Network	100 Mbps	1 Gbps	Fast file transfers to multiple servers

Note: For production environments with 50+ servers, use recommended specifications.

☐ PR Servers (Production/Primary Environment)

☐ PR Servers (Production/Primary Environment)

Role: Runs live business applications serving real customers

These servers handle actual production traffic and customer requests. They must be highly available and performant.

Services Running:

1. ☕ Java Applications (Microservices)

Purpose: The actual business applications that serve customers

What it does:

- **Runs microservices** - Your business logic (JAR files)
- **Handles API requests** - Processes customer transactions
- **Processes data** - Business logic execution
- **Serves traffic** - Responds to user requests

Why Java?

- Enterprise-grade reliability
- Rich ecosystem of libraries
- Excellent performance
- Wide industry adoption

Example Services:

- User Management Service (port 8080)
 - Payment Processing Service (port 8081)
 - Notification Service (port 9090)
-

2. ☐ JVM (Java Virtual Machine)

Purpose: Runtime environment that executes Java applications

What it does:

- **Executes bytecode** - Runs compiled Java applications
- **Manages memory** - Allocates heap space (4-8 GB)
- **Garbage collection** - Automatically frees unused memory (G1GC)
- **Optimizes performance** - Just-in-time compilation
- **Monitors resources** - Tracks CPU and memory usage

Why JVM?

- Platform independence (write once, run anywhere)
- Automatic memory management
- High performance with JIT compilation
- Mature and stable

Configuration:

```
-Xms4g          # Minimum heap: 4 GB
-Xmx8g          # Maximum heap: 8 GB
-XX:+UseG1GC    # Use G1 Garbage Collector
```

3. ⚙️ Systemd (Service Management)

Purpose: Linux service manager that controls application lifecycle

What it does:

- **Starts services** - Launches Java applications on boot
- **Stops services** - Gracefully shuts down applications
- **Restarts on failure** - Automatically recovers from crashes
- **Monitors status** - Tracks if services are running
- **Manages dependencies** - Ensures services start in correct order
- **Logs output** - Captures application logs

Why Systemd?

- Standard on modern Linux distributions
- Reliable service management

- Automatic restart on failure
- Easy integration with monitoring tools

Example Service File:

```
[Unit]
Description=Payment Service
After=network.target

[Service]
Type=simple
User=appuser
ExecStart=/usr/bin/java -jar /opt/app/payment.jar
Restart=always

[Install]
WantedBy=multi-user.target
```

External Connections:

Connection	Purpose	Protocol	Port
PR Database	Persistent data storage (PostgreSQL)	TCP	5432
PR Redis	In-memory caching for performance	TCP	6379

Why Separate Database/Cache?

- **Performance:** Dedicated resources for data operations
- **Scalability:** Can scale independently
- **Security:** Network isolation
- **Maintenance:** Can update without affecting app servers

Resource Requirements:

Resource	Minimum	Recommended	Why?
Memory	12 GB	16-32 GB	8 GB for JVM heap + OS + buffers
Disk	20 GB	50-100 GB	Application, logs, temporary files
CPU	4 cores	8 cores	Handle concurrent requests
Network	1 Gbps	10 Gbps	High-traffic production environment

JVM Configuration:

```
JAVA_OPTS="-Xms4g -Xmx8g -XX:+UseG1GC -XX:MaxGCPauseMillis=200"
```

Parameter	Value	Purpose
<code>-Xms</code>	4 GB	Initial heap size (prevents resizing)
<code>-Xmx</code>	8 GB	Maximum heap size (prevents OutOfMemory)
<code>-XX:+UseG1GC</code>	G1GC	Low-latency garbage collector
<code>-XX:MaxGCPauseMillis</code>	200ms	Target pause time for GC

DR Servers (Disaster Recovery Environment)

DR Servers (Disaster Recovery Environment)

Role: Backup environment that takes over if PR fails

DR servers are identical to PR servers but serve as a safety net. They ensure business continuity during disasters, outages, or maintenance.

Services Running:

1. ☕ Java Applications (Microservices)

Purpose: Identical business applications ready for failover

What it does:

- **Runs microservices** - Same applications as PR
- **Handles failover** - Takes over if PR environment fails
- **Processes requests** - Serves customers during PR outages
- **Maintains sync** - Keeps code in sync with PR

Why Separate DR?

- **Business continuity:** Operations continue during PR failures
- **Zero downtime:** Instant failover capability
- **Testing:** Safe environment for testing deployments
- **Compliance:** Many industries require DR environments

Failover Scenarios:

- PR datacenter outage
 - Network failures
 - Hardware failures
 - Planned maintenance
-

2. ☐ JVM (Java Virtual Machine)

Purpose: Runtime environment identical to PR

What it does:

- **Executes bytecode** - Runs Java applications
- **Manages memory** - Same heap configuration as PR (4-8 GB)
- **Garbage collection** - G1GC for consistent performance
- **Optimizes performance** - JIT compilation

Configuration: Identical to PR for consistent behavior

3. ⚙️ Systemd (Service Management)

Purpose: Manages application lifecycle

What it does:

- **Starts services** - Launches applications on boot
- **Stops services** - Graceful shutdown

- **Restarts on failure** - Automatic recovery
- **Monitors status** - Health checks
- **Ensures availability** - Always ready for failover

External Connections:

Connection	Purpose	Protocol	Port
DR Database	Persistent data storage (PostgreSQL)	TCP	5432
DR Redis	In-memory caching for performance	TCP	6379

Why Separate DR Database/Cache?

- **Data isolation:** PR and DR data remain separate
- **Independent operation:** DR can run without PR
- **Data replication:** Can sync from PR database
- **Testing:** Safe to test without affecting PR data

Resource Requirements:

Same as PR Servers - Ensures identical performance during failover

Resource	Minimum	Recommended	Why?
Memory	12 GB	16-32 GB	Match PR capacity
Disk	20 GB	50-100 GB	Same as PR
CPU	4 cores	8 cores	Handle full load during failover
Network	1 Gbps	10 Gbps	Support production traffic

JVM Configuration:

Identical to PR:

```
JAVA_OPTS="-Xms4g -Xmx8g -XX:+UseG1GC -XX:MaxGCPauseMillis=200"
```

▣ Separate Infrastructure Components

▣ Separate Infrastructure Components

These components run on dedicated servers, separate from application servers, for optimal performance and security.

1. ▣ PR Database Server (Separate Machine)

Purpose: Persistent data storage for Production environment

What it does:

- **Stores application data** - Customer records, transactions, configurations
- **Handles queries** - Processes SQL queries from PR Java applications
- **Maintains integrity** - ACID transactions ensure data consistency
- **Provides backups** - Regular automated backups
- **Manages connections** - Connection pooling for performance

Technology: PostgreSQL (recommended) or MySQL/Oracle

Why Separate?

- **Performance:** Dedicated CPU/memory for database operations
- **Scalability:** Can scale database independently
- **Security:** Network isolation and access control
- **Backup:** Easier to backup and restore
- **Maintenance:** Update database without affecting app servers

Resource Requirements:

- **Memory:** 16-32+ GB (for caching and buffers)
 - **Disk:** 100-500+ GB (depends on data volume)
 - **CPU:** 4-8 cores
 - **Storage:** SSD for better I/O performance
-

2. ▣ DR Database Server (Separate Machine)

Purpose: Persistent data storage for Disaster Recovery environment

What it does:

- **Stores DR data** - Separate database for DR environment
- **Handles DR queries** - Processes queries from DR applications

- **Maintains integrity** - ACID transactions
- **Replication** - Can replicate from PR database (optional)
- **Failover ready** - Takes over if PR database fails

Why Separate from PR Database?

- **Independence:** DR can operate without PR
- **Testing:** Safe to test without affecting production data
- **Isolation:** Failures in PR don't affect DR
- **Compliance:** Separate environments for audit requirements

Resource Requirements: Same as PR Database

3. ■ PR Redis Server (Separate Machine)

Purpose: In-memory caching for Production environment

What it does:

- **Caches data** - Stores frequently accessed data in memory
- **Improves performance** - Reduces database load by 70-90%
- **Stores sessions** - User session management
- **Reduces latency** - Sub-millisecond response times
- **Handles pub/sub** - Real-time messaging between services

Why Redis?

- **Speed:** In-memory storage = ultra-fast access
- **Versatility:** Cache, session store, message broker
- **Persistence:** Optional disk persistence
- **Clustering:** Can scale horizontally

Why Separate?

- **Performance:** Dedicated memory for caching
- **Scalability:** Scale cache independently
- **Isolation:** Cache failures don't affect app servers

Resource Requirements:

- **Memory:** 8-16+ GB (all data stored in RAM)
- **Disk:** 20-50 GB (for persistence)
- **CPU:** 2-4 cores
- **Network:** Low latency connection to app servers

Example Use Cases:

- Cache database query results
- Store user sessions

- Rate limiting
- Real-time analytics

4. DR Redis Server (Separate Machine)

Purpose: In-memory caching for Disaster Recovery environment

What it does:

- **Caches DR data** - Separate cache for DR environment
- **Improves DR performance** - Same benefits as PR Redis
- **Stores DR sessions** - Independent session management
- **Failover ready** - Takes over if PR Redis fails

Why Separate from PR Redis?

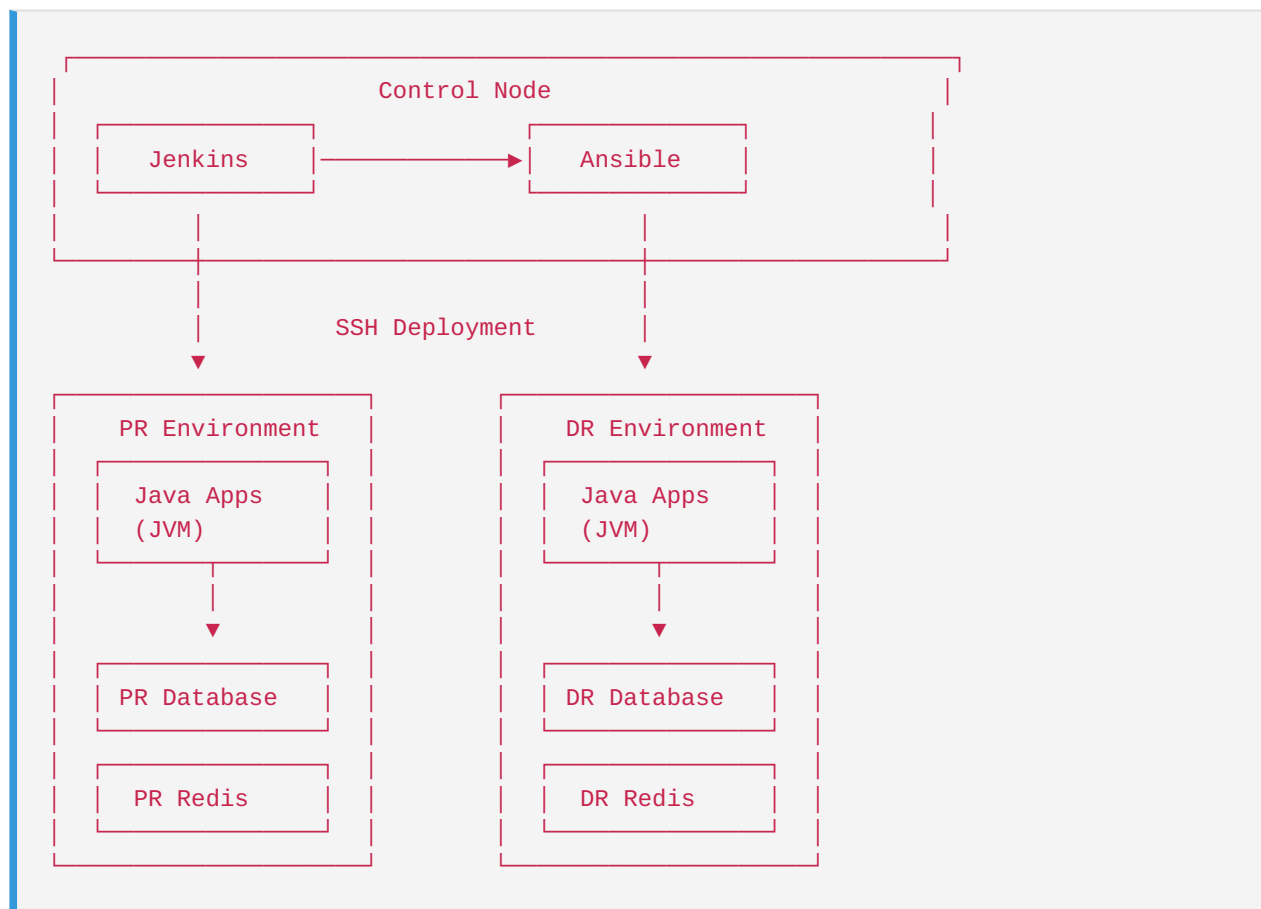
- **Independence:** DR cache operates independently
- **Testing:** Safe to test without affecting PR cache
- **Isolation:** PR cache failures don't affect DR

Resource Requirements: Same as PR Redis

Summary Table

Node	Services	Min RAM	Rec RAM	Min Disk	Rec Disk
Control Node	Jenkins + Ansible	8 GB	16 GB	20 GB	50 GB
PR Servers	Java Apps (microservices)	12 GB	16-32 GB	20 GB	50-100 GB
DR Servers	Java Apps (microservices)	12 GB	16-32 GB	20 GB	50-100 GB
Database Servers	PostgreSQL (separate)	16 GB	32+ GB	100 GB	500+ GB
Cache Servers	Redis (separate)	8 GB	16+ GB	20 GB	50+ GB

Architecture Diagram



Deployment Flow

1. **Jenkins** (Control Node) triggers deployment
2. **Ansible** (Control Node) executes playbooks
3. **Ansible** deploys to **PR Servers**
4. **PR Servers** run Java applications
5. **Ansible** validates PR deployment
6. **Ansible** deploys to **DR Servers**
7. **DR Servers** run Java applications
8. **Jenkins** reports deployment status

Network Requirements

▮ Network Requirements

Control Node Connectivity:

Destination	Protocol	Port	Purpose
All PR Servers	SSH	22	Deploy and manage applications
All DR Servers	SSH	22	Deploy and manage applications
Git Repository	HTTPS/SSH	443/22	Pull deployment code
SMTP Server	SMTP	25/587	Send email notifications

Security Requirements:

- SSH key-based authentication (no passwords)
 - Firewall rules allowing outbound connections
 - Network access to all target servers
-

PR Servers Connectivity:

Destination	Protocol	Port	Purpose
PR Database	PostgreSQL	5432	Database queries
PR Redis	Redis	6379	Cache operations
Load Balancer	HTTP/HTTPS	80/443	Receive customer traffic

Application Ports:

- 8080 - User Management Service
 - 8081 - Payment Processing Service
 - 9090 - Notification Service
-

DR Servers Connectivity:

Destination	Protocol	Port	Purpose
DR Database	PostgreSQL	5432	Database queries
DR Redis	Redis	6379	Cache operations
Load Balancer	HTTP/HTTPS	80/443	Receive failover traffic

Application Ports: Same as PR (8080, 8081, 9090)

Minimum vs Recommended Specifications

For Testing/Development:

Use Case: Learning, testing, small-scale deployments

Component	RAM	Disk	Servers	Total Cost
Control Node	8 GB	20 GB	1	Low
PR Servers	12 GB	20 GB	1	Low
DR Servers	12 GB	20 GB	1	Low
Database	16 GB	100 GB	1 (shared)	Medium
Redis	8 GB	20 GB	1 (shared)	Low

Total: 5 servers, suitable for testing

For Production:

Use Case: Live customer-facing applications, high availability

Component	RAM	Disk	Servers	Total Cost
Control Node	16 GB	50 GB	1	Medium
PR Servers	32 GB	100 GB	3-5	High
DR Servers	32 GB	100 GB	3-5	High
PR Database	32+ GB	500+ GB	1	High
DR Database	32+ GB	500+ GB	1	High
PR Redis	16+ GB	50+ GB	1	Medium
DR Redis	16+ GB	50+ GB	1	Medium

Total: 11-15 servers, suitable for production with high availability

□ Why This Architecture?

1. □ Separation of Concerns

Control Node (Deployment Management)

- Manages deployments but doesn't run business applications
- Isolated from production traffic
- Can be updated without affecting applications
- Single point of control for all deployments

PR/DR Servers (Application Execution)

- Only run business applications
- No deployment tools installed
- Optimized for application performance
- Clean separation of responsibilities

Databases and Caches (Data Layer)

- Isolated for performance and security
- Dedicated resources for data operations
- Can be scaled independently
- Network-level security controls

Benefits:

- ☐ Easier troubleshooting (clear boundaries)
 - ☐ Better security (limited access)
 - ☐ Independent scaling
 - ☐ Simplified maintenance
-

2. ☐ High Availability

PR (Production) Environment

- Handles live customer traffic
- Multiple servers for load balancing
- Optimized for performance

DR (Disaster Recovery) Environment

- Identical setup to PR
- Ready for instant failover
- Ensures business continuity

Failover Scenarios:

- ☐ Datacenter outage → Switch to DR
- ☐ Network failure → DR takes over
- ☐ Planned maintenance → Move traffic to DR
- ☐ Hardware failure → Automatic failover

Benefits:

- ☐ 99.9%+ uptime
 - ☐ Zero data loss
 - ☐ Minimal downtime
 - ☐ Customer trust
-

3. ☐ Automation

Jenkins + Ansible Combination**Manual Deployment Problems:**

- ☐ Human errors (typos, wrong files)
- ☐ Inconsistent deployments
- ☐ Time-consuming (hours)
- ☐ No audit trail
- ☐ Difficult rollbacks

Automated Deployment Benefits:

- ☐ Consistent every time
- ☐ Fast (8-10 minutes)
- ☐ Complete audit trail
- ☐ One-click rollback
- ☐ Reduced errors by 95%

Example:

Manual: 2 hours, 30% error rate
Automated: 10 minutes, <1% error rate

4. ☐ Scalability

Horizontal Scaling (Add more servers)

- Add PR servers → Handle more traffic
- Add DR servers → Better failover capacity
- Add database replicas → Faster queries
- Add Redis nodes → More cache capacity

Vertical Scaling (Bigger servers)

- Increase RAM → Handle more concurrent users
- Increase CPU → Faster processing
- Increase disk → Store more data

Independent Scaling:

- Scale applications without touching database
- Scale cache without affecting applications
- Scale Control Node for more deployments

Benefits:

- ☐ Handle traffic growth
- ☐ Cost-effective (scale what you need)
- ☐ No downtime during scaling
- ☐ Future-proof architecture

Getting Started Checklist

Phase 1: Infrastructure Setup

- ☐ Provision Control Node server
- ☐ Provision PR application servers
- ☐ Provision DR application servers
- ☐ Provision database servers
- ☐ Provision Redis servers
- ☐ Configure network connectivity
- ☐ Set up firewalls and security groups

Phase 2: Control Node Configuration

- ☐ Install Jenkins
- ☐ Install Ansible
- ☐ Install Python 3.x
- ☐ Configure SSH keys
- ☐ Clone deployment repository
- ☐ Configure Jenkins pipeline

Phase 3: Application Server Setup

- ☐ Install Java/JVM
- ☐ Configure systemd services
- ☐ Set up application directories
- ☐ Configure JVM parameters
- ☐ Test SSH connectivity from Control Node

Phase 4: Database & Cache Setup

- ☐ Install PostgreSQL on database servers
- ☐ Install Redis on cache servers
- ☐ Configure database connections
- ☐ Configure Redis connections
- ☐ Set up backups

Phase 5: Testing

- ☐ Test deployment to PR
 - ☐ Test deployment to DR
 - ☐ Test rollback functionality
 - ☐ Test failover scenarios
 - ☐ Load testing
-

Support and Documentation

Additional Resources:

- Setup Guide: /docs/SETUP_GUIDE.md
 - Deployment Guide: /docs/DEPLOYMENT_GUIDE.md
 - Troubleshooting: </docs/TROUBLESHOOTING.md>
 - Production Checklist: </PRODUCTION-READY.md>
-

Document Version: 2.0

Last Updated: May 14, 2026

Location: </home/ubuntu/Automate-Patch-Release/docs/>

Maintained By: DevOps Team